

SMART RESUME PARSER PROJECT

Internship Project Report + Full Python Program

Based on the given instructions: choose a project from the list, build it using any tools, focus on clarity for interviews, and prepare a 1–2 page report PDF containing Introduction, Abstract, Tools Used, Steps Involved, and Conclusion. This document includes those report sections and also provides the complete Python program.

1. Introduction

The Smart Resume Parser is a Python-based project that reads resumes in PDF and DOCX format and extracts structured details such as candidate name, email, phone number, skills, education, and experience. This project is useful for recruiters, HR teams, and students who want to automate resume screening. It follows the project list exactly by using Python, spaCy, PyMuPDF, docx, and Streamlit.

2. Abstract

This project converts unstructured resume files into clean structured data. The system first reads the resume content, preprocesses the extracted text, applies simple natural language processing and regular expressions, and then organizes the result into JSON and CSV files. A user-friendly Streamlit interface is added so that any user can upload a resume and instantly view parsed information.

3. Tools Used

Tool / Library	Purpose
Python	Main programming language used to build the project
PyMuPDF (fitz)	Extract text from PDF resumes
python-docx	Extract text from DOCX resumes
spaCy	Detect entities like candidate name and support NLP processing
re (Regex)	Extract phone, email, and skill matches
Pandas	Save structured output in CSV format
JSON	Store parsed resume data in machine-readable format
Streamlit	Create a simple web UI for uploading and viewing resumes

4. Steps Involved in Building the Project

- 1 Create the project folder and install required libraries.
- 2 Load PDF or DOCX resume files from the user.
- 3 Extract text using PyMuPDF for PDF and python-docx for DOCX.
- 4 Clean the extracted text by removing extra spaces and symbols.
- 5 Use regex to identify email and phone number.
- 6 Use spaCy to detect the candidate name and improve text understanding.
- 7 Match skills from a predefined skills database.
- 8 Identify education and experience lines using keyword-based logic.
- 9 Store the extracted information in a Python dictionary.
- 10 Export the final result into JSON and CSV files.

- 11 Build a Streamlit interface for upload and result display.
- 12 Test the project with at least 5 resumes as mentioned in deliverables.

5. Conclusion

The Smart Resume Parser is a practical internship project because it combines file handling, text extraction, natural language processing, regular expressions, data storage, and UI design in one application. It is easy to explain in interviews and can be improved further by adding resume scoring, job-role matching, database storage, and skill recommendation features.

Deliverables Included

This submission contains: complete Python code, Streamlit UI logic, export to JSON and CSV, and a report that matches the required headings from the internship guidelines.

FULL PYTHON PROGRAM

Save the following code as **smart_resume_parser_project.py** and run it with Streamlit.

```
"""
Smart Resume Parser Project
Internship project based on the given project-list instructions.

Features:
- Extracts text from PDF and DOCX resumes
- Cleans text
- Extracts:
* Name
* Email
* Phone
* Skills
* Education
* Experience
- Saves output to JSON and CSV
- Includes a simple Streamlit UI

Install:
pip install streamlit spacy pymupdf python-docx pandas
python -m spacy download en_core_web_sm

Run:
streamlit run smart_resume_parser_project.py
"""

from __future__ import annotations

import re
import json
from pathlib import Path
from typing import Dict, List, Any

import fitz # PyMuPDF
import docx
import pandas as pd
import spacy
import streamlit as st

# -----
# Configuration
# -----
SKILLS_DB = [
    "python", "java", "c", "c++", "html", "css", "javascript", "sql",
    "machine learning", "deep learning", "data analysis", "data science",
    "pandas", "numpy", "matplotlib", "power bi", "excel", "streamlit",
    "flask", "django", "nlp", "spacy", "tensorflow", "pytorch",
    "git", "github", "mysql", "mongodb", "opencv", "communication",
    "problem solving", "teamwork"
]

EDUCATION_KEYWORDS = [
    "b.e", "b.tech", "m.e", "m.tech", "b.sc", "m.sc", "bca", "mca",
    "b.com", "mba", "degree", "university", "college", "school",
    "higher secondary", "sslc", "hsc"
]

EXPERIENCE_KEYWORDS = [
    "experience", "internship", "intern", "project", "worked", "developer",
    "engineer", "analyst", "responsibilities", "employment"
]

EMAIL_PATTERN = r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}"
PHONE_PATTERN = r"(?:\+91[-\s]?)?[6-9]\d{9}"
```

```
@st.cache_resource
def load_nlp():
    return spacy.load("en_core_web_sm")
```

```
def extract_text_from_pdf(file_path: Path) -> str:
    text = []
    with fitz.open(file_path) as pdf:
        for page in pdf:
            text.append(page.get_text())
    return "\n".join(text)
```

```
def extract_text_from_docx(file_path: Path) -> str:
    document = docx.Document(file_path)
    text = [para.text for para in document.paragraphs]
    return "\n".join(text)
```

```
def extract_text(file_path: Path) -> str:
    suffix = file_path.suffix.lower()
    if suffix == ".pdf":
        return extract_text_from_pdf(file_path)
    if suffix == ".docx":
        return extract_text_from_docx(file_path)
    raise ValueError("Unsupported file format. Please upload PDF or DOCX.")
```

```
def clean_text(text: str) -> str:
    text = re.sub(r"\s+", " ", text)
    text = re.sub(r"[•|■|●]", " ", text)
    return text.strip()
```

```
def extract_name(text: str, nlp) -> str:
    doc = nlp(text[:1000])
    for ent in doc.ents:
        if ent.label_ == "PERSON" and len(ent.text.split()) <= 4:
            return ent.text.title()
    lines = text.split(".")
    if lines:
        first_line = lines[0].strip()
        if 2 <= len(first_line.split()) <= 4:
            return first_line.title()
    return "Not found"
```

```
def extract_email(text: str) -> str:
    match = re.search(EMAIL_PATTERN, text)
    return match.group(0) if match else "Not found"
```

```
def extract_phone(text: str) -> str:
    match = re.search(PHONE_PATTERN, text)
    return match.group(0) if match else "Not found"
```

```
def extract_skills(text: str) -> List[str]:
    lower_text = text.lower()
    found_skills = []
    for skill in SKILLS_DB:
        if re.search(r"\b" + re.escape(skill) + r"\b", lower_text):
            found_skills.append(skill.title())
    return sorted(set(found_skills))
```

```

def extract_education(text: str) -> List[str]:
    sentences = re.split(r"(?<=[.!?])\s+", text)
    result = []
    for sentence in sentences:
        if any(keyword in sentence.lower() for keyword in EDUCATION_KEYWORDS):
            result.append(sentence.strip())
    return result[:10]

def extract_experience(text: str) -> List[str]:
    sentences = re.split(r"(?<=[.!?])\s+", text)
    result = []
    for sentence in sentences:
        if any(keyword in sentence.lower() for keyword in EXPERIENCE_KEYWORDS):
            result.append(sentence.strip())
    return result[:10]

def parse_resume(file_path: Path, nlp) -> Dict[str, Any]:
    raw_text = extract_text(file_path)
    cleaned = clean_text(raw_text)

    parsed_data = {
        "file_name": file_path.name,
        "name": extract_name(cleaned, nlp),
        "email": extract_email(cleaned),
        "phone": extract_phone(cleaned),
        "skills": extract_skills(cleaned),
        "education": extract_education(cleaned),
        "experience": extract_experience(cleaned),
        "raw_text_preview": cleaned[:500] + "..." if len(cleaned) > 500 else cleaned,
    }
    return parsed_data

def save_to_json(data: Dict[str, Any], output_path: Path) -> None:
    with open(output_path, "w", encoding="utf-8") as f:
        json.dump(data, f, indent=4, ensure_ascii=False)

def save_to_csv(data: Dict[str, Any], output_path: Path) -> None:
    row = {
        "file_name": data["file_name"],
        "name": data["name"],
        "email": data["email"],
        "phone": data["phone"],
        "skills": ", ".join(data["skills"]),
        "education": " | ".join(data["education"]),
        "experience": " | ".join(data["experience"]),
    }
    df = pd.DataFrame([row])
    df.to_csv(output_path, index=False)

def main():
    st.set_page_config(page_title="Smart Resume Parser", page_icon="📄", layout="wide")
    st.title("📄 Smart Resume Parser")
    st.write("Upload a PDF or DOCX resume to extract structured details.")

    nlp = load_nlp()
    uploaded_file = st.file_uploader("Upload Resume", type=["pdf", "docx"])

    if uploaded_file is not None:
        upload_dir = Path("uploaded_resumes")
        output_dir = Path("output_files")
        upload_dir.mkdir(exist_ok=True)

```

```
output_dir.mkdir(exist_ok=True)

saved_path = upload_dir / uploaded_file.name
with open(saved_path, "wb") as f:
    f.write(uploaded_file.getbuffer())

try:
    result = parse_resume(saved_path, nlp)

    st.subheader("Extracted Information")
    st.json(result)

    st.subheader("Quick View")
    st.write(f"**Name:** {result['name']}")
    st.write(f"**Email:** {result['email']}")
    st.write(f"**Phone:** {result['phone']}")
    st.write(f"**Skills:** {' '.join(result['skills']) if result['skills'] else 'Not found'}")

    json_path = output_dir / f"{saved_path.stem}.json"
    csv_path = output_dir / f"{saved_path.stem}.csv"

    save_to_json(result, json_path)
    save_to_csv(result, csv_path)

    st.success("Parsing completed successfully.")
    st.write(f"JSON saved to: {json_path}")
    st.write(f"CSV saved to: {csv_path}")

except Exception as e:
    st.error(f"Error while parsing resume: {e}")

if __name__ == "__main__":
    main()
```